



Competitive Programming Handbook



Table of Contents

1. General

- [1.1 Template](#)
- [1.2 Math Mod Space](#)
- [1.3 Stress Test Code](#)

2. Graphs

- [2.1 Basic Algorithms](#)
- [2.2 DSU](#)
- [2.3 DSU with Rollback](#)
- [2.4 MST](#)
- [2.5 DSU on Trees](#)
- [2.6 LCA \(Binary Lifting\)](#)
- [2.7 LCA \(Sparse Table\)](#)
- [2.8 LCA \(Offline Tarjan\)](#)
- [2.9 Tarjan SCC](#)
- [2.10 Tarjan Bridges](#)
- [2.11 Tarjan Articulation Points](#)
- [2.12 Max Flow](#)
- [2.13 LCA Mahmoud](#)
- [2.14 Dijkstra On Grid](#)
- [2.15 BiMatching](#)
- [2.16 HLD](#)
- [2.17 SACK](#)

3. DP

- [3.1 LIS](#)
- [3.2 LIS With Data Structures](#)
- [3.3 Digits](#)

4. Math

- [4.1 Matrix Exponentiation](#)
- [4.2 Big Int Mul](#)
- [4.3 Exact Match but multi with num chars](#)
- [4.4 FFT](#)
- [4.5 FFT Mod](#)
- [4.6 FWHT](#)
- [4.7 NTT](#)
- [4.8 Pow](#)

- [4.9 String Matching With Wild Cards](#)

5. Geometry

- [5.1 Geometry](#)

6. Data Structures

- [6.1 Segment Tree](#)
- [6.2 Lazy Segment Tree](#)
- [6.3 BIT](#)
- [6.4 Range Update BIT](#)
- [6.5 Sqrt Decomposition](#)
- [6.6 Mo's Algorithm](#)
- [6.7 Segment Tree With Pointers](#)
- [6.8 Segment Tree Mahmoud](#)

7. Strings

- [7.1 Multiset Hashing](#)
- [7.2 Rolling Hashing](#)
- [7.3 KMP](#)
- [7.4 Trie](#)
- [7.5 Binary Trie](#)

8. Game Theory

- [8.1 Grundy Numbers](#)
- [8.2 MEX in \$O\(\log\(n\)^2\)\$](#)
- [8.3 MEX in \$O\(\log\(n\)\)\$](#)

1. General

1.1 Template

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define Kero
\
    ios_base::sync_with_stdio(0);
\
    cout.tie(0);
\
    cin.tie(0)
#define ll long long
#define ull unsigned long long
#define all(x) x.begin(), x.end()
#define allr(x) x.rbegin(), x.rend()
```

```

#define int ll
#define ld long double
#define pii pair<int, int>
#define endl '\n'
#define see(x) " [" << #x << " = " << (x) << "]"
using namespace std;
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag,
tree_order_statistics_node_update>;
// order_of_key (k) : Number of items strictly smaller than k
// find_by_order(k) : K-th element in a set (counting from zero)
const int INF = 1e18, N = 2e5 + 5, mod = 1e9 + 7;
const ld pi = 2 * acos(0), eps = 1e-6;
typedef __int128_t lll;

void fileIO()
{
#ifdef ONLINE_JUDGE
    freopen("io/input.txt", "r", stdin);
    freopen("io/output.txt", "w", stdout);
#endif
}

```

1.2 Math Mod Space

```

map<int, int> facts;
int addm(int a)
{
    return ((a % mod) + mod) % mod;
}
template <typename... Args>
int addm(int a, Args... args)
{
    return (addm(a) + addm(args...)) % mod;
}
int subtm(int a, int b)
{
    return ((a % mod) - (b % mod) + mod) % mod;
}
int mul(int a)
{
    return ((a % mod) + mod) % mod;
}
template <typename... Args>
int mul(int a, Args... args)
{
    return (mul(a) * mul(args...)) % mod;
}

```

```

}
int fastpow(int b, int p)
{
    if (p == 0 || b == 1)
        return 1;
    int sub = fastpow(b, p / 2);
    if (p % 2 == 0)
        return mul(sub, sub);
    return mul(mul(sub, sub), b);
}
int modInverse(int a)
{
    return fastpow(a, mod - 2);
}
int divi(int a, int b)
{
    return mul(a, modInverse(b));
}
ll fact(ll a)
{
    if (a == 0ll)
        return 1;
    if (!facts.count(a))
        facts[a] = mul(fact(a - 1), a);
    return facts[a];
}
int nPr(int n, int r)
{
    if (n < r)
        return 0;
    return divi(fact(n), fact(n - r));
}
int nCr(int n, int r)
{
    if (n < r)
        return 0;
    return divi(nPr(n, r), fact(r));
}
int starsNbars(int n, int r)
{
    return nCr(n + r - 1, r);
}

```

1.3 Stress Test Code

```

#include <bits/stdc++.h>

using namespace std;
#define pii pair<int, int>
// Random test case generator (writes input to in.txt)
void generate_test_case()

```

```

{
    ofstream fout("io/input.txt");
    fout << 1 << "\n";
    int N = (rand() % int(2e3)) + 1;
    fout << N << '\n';
    for (int i = 1; i <= N; i++)
        fout << (rand() % N) + 1 << ' ';
    fout << '\n';
    // vector<pii> edges;
    for (int i = 2; i <= N; i++)
    {
        int parent = (rand() % (i - 1)) + 1;

        fout << parent << ' ' << i << '\n';
    }
    fout.close();
}

// Read entire file into a string (for output comparison)
string read_file(const string &filename)
{
    ifstream fin(filename);
    stringstream buffer;
    buffer << fin.rdbuf();
    return buffer.str();
}

int main()
{
    srand(time(0));
    int test_num = 0;

    for(; test_num < 1000;)
    {
        test_num++;
        generate_test_case();

        // Run both solutions on the same input
        system("./build/naive");
        system("./build/fast");
        // Compare outputs
        string out = read_file("io/output.txt");
        string brute = read_file("io/output_naive.txt");

        if (out != brute)
        {
            cout << "Test " << test_num << " FAILED!\n";
            // cout << "Input:\n" << read_file("io/input.txt") << "\n";
            // cout << "Fast output:\n" << out << "\n";
            // cout << "Brute output:\n" << brute << "\n";
            break;
        }
        else
        {

```

```
        cout << "Test " << test_num << " passed.\n";
    }
}

return 0;
}
```

2. Graphs

2.1 Basic Algorithms

```
bool visited[N]{};
void dfs(vector<vector<int>> &graph, int st)
{
    visited[st] = true;
    for (auto &child : graph[st])
        if (!visited[child])
            dfs(graph, child);
}

vector<int> bfs(vector<vector<int>> &graph, int st)
{
    vector<int> dist(graph.size(), INF);
    queue<int> nextToVisit;
    dist[st] = 0;
    nextToVisit.push(st);
    while (!nextToVisit.empty())
    {
        auto cur = nextToVisit.front();
        nextToVisit.pop();
        for (auto &ch : graph[cur])
        {
            if (dist[ch] == INF)
            {
                dist[ch] = dist[cur] + 1;
                nextToVisit.push(ch);
            }
        }
    }
    return dist;
}

vector<int> dijk(vector<vector<pii>> &graph, int st)
{
    vector<int> dist(graph.size(), INF);
    priority_queue<pair<int, int>, vector<pair<int, int>>,
greater<pair<int, int>>> nextToVisit;
    dist[st] = 0;
    nextToVisit.push({0, st});
    while (nextToVisit.size())
```

```

{
    auto [curCost, curDist] = nextToVisit.top();
    nextToVisit.pop();
    if (curCost > dist[curDist])
        continue;
    for (auto &[cw, cd] : graph[curDist])
    {
        if (dist[cd] > cw + curCost)
        {
            dist[cd] = cw + curCost;
            nextToVisit.push({dist[cd], cd});
        }
    }
}
return dist;
}

```

```

void kahn(vector<vector<int>> &graph, int &n)
{
    vector<int> degree(n + 1, 0);
    vector<bool> visited(n + 1, false);
    for (int i = 1; i <= n; i++)
    {
        for (auto &j : graph[i])
        {
            degree[j]++;
        }
    }
    queue<int> nextToVisit;
    for (int i = 1; i <= n; i++)
    {
        if (!degree[i])
        {
            nextToVisit.push(i);
            visited[i] = true;
        }
    }
    vector<int> result;
    while (nextToVisit.size())
    {
        int cur = nextToVisit.front();
        nextToVisit.pop();
        result.push_back(cur);
        for (auto &j : graph[cur])
        {
            degree[j]--;
            if (!degree[j])
                nextToVisit.push(j);
        }
    }
    if (result.size() != n)
    {
        cout << "IMPOSSIBLE" << endl;
    }
}

```

```

    else
    {
        for (auto &j : result)
            cout << j << ' ';
        cout << endl;
    }
}

vector<int> BellmanFord(vector<vector<pii>> &graph, int &V, int st)
{
    vector<int> dist(V + 1, INF);
    dist[st] = 0;
    bool flag = false;
    for (int edgesUsed = 1; edgesUsed <= V; edgesUsed++)
    {
        for (int u = 1; u <= V; u++)
        {
            if (dist[u] == INF)
                continue;
            for (auto &[cost, v] : graph[u])
            {
                if(dist[v] > dist[u] + cost)
                    dist[v] = dist[u] + cost, flag |= (v == V and edgesUsed
== V);
            }
        }
    }
    if(flag)
        return {};
    return dist;
}

void floyd()
{
    int d[N][N];
    for(int k = 1; k <= n; k++)
        for(int u = 1; u <= n; u++)
            for(int v = 1; v <= n; v++)
                d[u][v] = max(d[u][v], d[u][k] + d[k][v]);
}

```

2.2 DSU

```

struct DSU
{
    vector<int> par, sz;
    DSU(int n)
    {
        sz = par = vector<int>(n + 1, 1);
        iota(par.begin(), par.end(), 0);
    }
    void merge(int u, int v)

```

```

{
    u = getPar(u);
    v = getPar(v);
    if (sz[v] > sz[u])
        swap(v, u);
    par[v] = u;
    sz[u] += sz[v];
}
int getPar(int u)
{
    if (par[u] == u)
        return u;
    return par[u] = getPar(par[u]);
}
bool get(int u, int v)
{
    return (getPar(u) == getPar(v));
}
};

```

2.3 DSU with Rollback

```

struct DSU
{
    vector<int> par, sz;
    stack<pair<int &, int>> rollbacks;
    DSU(int n)
    {
        par = sz = vector<int>(n + 1, 1);
        iota(all(par), 0);
    }
    int getPar(int u)
    {
        if (par[u] == u)
            return u;
        return getPar(par[u]); // No path compression due to rollbacks
    }
    void merge(int u, int v)
    {
        u = getPar(u);
        v = getPar(v);
        rollbacks.push({par[u], par[u]});
        rollbacks.push({par[v], par[v]});
        rollbacks.push({sz[u], sz[u]});
        rollbacks.push({sz[v], sz[v]});
        if (u == v)
            return;
        if (sz[v] > sz[u])
            swap(v, u);
        sz[u] += sz[v];
        par[v] = u;
    }
};

```

```

    }
    void rollback()
    {
        assert(rollbacks.size() >= 4);
        for (int i = 0; i < 4; i++)
        {
            auto [variable, value] = rollbacks.top();
            variable = value;
            rollbacks.pop();
        }
    }
    int get(int u, int v)
    {
        return getPar(u) == getPar(v);
    }
};

```

2.4 MST

```

DSU dsu(n);
vector<edge> edges;
for (int i = 0; i < m; i++)
{
    int c, u, v;
    cin >> u >> v >> c;
    edges.push_back({c, u, v});
}
sort(all(edges));
int sum = 0;
for (auto &[c, u, v] : edges)
{
    if (dsu.get(u, v))
        continue;
    sum += c;
    dsu.merge(u, v);
}

```

2.5 DSU on Trees

```

int c[N];
vector<int> graph[N];
set<int> s[N];
int ans[N];
void dfs(int cur, int par)
{
    s[cur].insert(c[cur]);
    for (auto &ch : graph[cur])
    {
        if (ch == par)

```

```

        continue;
    dfs(ch, cur);
    if(s[ch].size() > s[cur].size())
        swap(s[ch], s[cur]);
    for(auto &x : s[ch])
        s[cur].insert(x);
}
ans[cur] = s[cur].size();
}

```

2.6 LCA (Binary Lifting)

```

const int N = 1e5 + 5, LG = 23;
vector<int> graph[N];
int up[LG][N];
int deep[N]{};
void dfs(int cur, int par = -1, int depth = 0)
{
    deep[cur] = depth;
    for(auto &ch : graph[cur])
    {
        if(ch == par)
            continue;
        up[0][ch] = cur;
        for(int lg = 1; lg < LG and ~up[lg - 1][ch]; lg++)
            up[lg][ch] = up[lg - 1][up[lg - 1][ch]];
        dfs(ch, cur, depth + 1);
    }
}
int kthAnc(int u, int k)
{
    for(int lg = 0; lg < LG and ~u; lg++)
        if((k >> lg) & 1)
            u = up[lg][u];
    return u;
}
int getLca(int u, int v)
{
    if(deep[v] > deep[u])
        swap(v, u);
    u = kthAnc(u, deep[u] - deep[v]);
    if(u == v)
        return u;
    for(int lg = LG - 1; lg >= 0; --lg)
        if(up[lg][u] != up[lg][v])
            u = up[lg][u], v = up[lg][v];
    return up[0][u];
}

```

2.7 LCA (Sparse Table)

```

template <typename T>
struct SparseTable
{
    vector<vector<T>> table;
    vector<int> lgs;
    function<T(T, T)> myFunction;

    void build(T arr[], function<T(T, T)> fun, int n)
    {
        myFunction = fun;
        table = vector<vector<T>>(n + 1, vector<T>(20));
        lgs = vector<int>(n + 1);
        for (int i = 0; i < n; i++)
            table[i][0] = arr[i];
        for (int i = 2; i <= n; i++)
            lgs[i] = lgs[i / 2] + 1;
        for (int lg = 1; lg <= lgs[n]; lg++)
            for (int i = 0; i + (1 << lg) - 1 < n; i++)
                table[i][lg] = myFunction(table[i][lg - 1], table[i + (1 <<
(lg - 1))][lg - 1]);
    }
    T get(int l, int r)
    {
        return myFunction(table[l][lgs[r - l + 1]], table[r - (1 << lgs[r -
l + 1]) + 1][lgs[r - l + 1]]);
    }
};

SparseTable<int> sp;
vector<int> graph[N];
int first[N];
int eular[2 * N];
int deep[N];
int timer = 0;
void eular_tour(int cur, int par = -1, int depth = 0)
{
    deep[cur] = depth;
    eular[timer] = cur;
    first[cur] = timer++;
    for (auto &ch : graph[cur])
        if (ch != par)
            eular_tour(ch, cur, depth + 1), eular[timer++] = cur;
}
#define getLca(x, y) sp.get(min(first[x], first[y]), max(first[x],
first[y]))

```

2.8 LCA (Offline Tarjan)

```

vector<int> g[N];
vector<pair<int, int>> queries[N]; // queries[u] contains {v, idx}
int ancestor[N], lca_answer[N];

```

```

bool visited[N];

void dfs(int u, int p, DSU &dsu) {
    ancestor[u] = u;
    visited[u] = true;

    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u, dsu);
        dsu.merge(u, v);
        ancestor[dsu.getPar(u)] = u;
    }

    for (auto [v, idx] : queries[u]) {
        if (visited[v]) {
            lca_answer[idx] = ancestor[dsu.getPar(v)];
        }
    }
}

```

2.9 Tarjan SCC

```

int tarjan(vector<vector<int>> &graph)
{
    int n = graph.size() - 1;
    vector<int> low(n + 1, -1), inStack(n + 1, 0), dfsn(n + 1, -1), comp(n
+ 1, -1);
    vector<vector<int>> comps;
    stack<int> st;
    int timer = 0;
    function<void(int)> dfs = [&](int cur) {
        low[cur] = dfsn[cur] = timer++;
        inStack[cur] = 1;
        st.push(cur);
        for (auto &ch : graph[cur])
        {
            if (dfsn[ch] == -1)
            {
                dfs(ch);
                low[cur] = min(low[cur], low[ch]);
            }
            else if (inStack[ch])
            {
                low[cur] = min(low[cur], dfsn[ch]);
            }
        }
        if (low[cur] == dfsn[cur])
        {
            comps.emplace_back(); // Add New Comp
            int node = -1;
            while (node != cur)

```

```

        {
            node = st.top();
            st.pop();
            inStack[node] = 0;
            comps.back().push_back(node);
            comp[node] = comps.back().size() - 1;
        }
    }

};

for (int i = 1; i <= n; i++)
    if (dfsn[i] == -1)
        dfs(i);
return (comps.size() == 1);
}

```

2.10 Tarjan Bridges

```

int tarjan(vector<vector<int>> &graph)
{
    int n = graph.size() - 1;
    vector<int> dfsn(n + 1, -1), low(n + 1, -1);
    int ret = 0;
    int timer = 0;
    set<pii> bridges;
    function<void(int, int)> dfs = [&](int cur, int par) {
        dfsn[cur] = low[cur] = timer++;
        for (auto &ch : graph[cur])
        {
            if (dfsn[ch] == -1)
            {
                dfs(ch, cur);
                low[cur] = min(low[cur], low[ch]);
                if (low[ch] == dfsn[ch])
                    bridges.insert({cur, ch});
            }
            else if (ch != par)
                low[cur] = min(low[cur], dfsn[ch]);
        }
    };
    dfs(1, -1);
    vector<int> comp(n + 1, -1);
    vector<vector<int>> bridgesTree(n + 1);
    timer = 1;
    function<void(int)> calcComp = [&](int cur) {
        for (auto &ch : graph[cur])
            if (comp[ch] == -1 and !bridges.count({cur, ch}) and
!bridges.count({ch, cur}))
                comp[ch] = comp[cur], calcComp(ch);
    };
    for (int i = 1; i <= n; i++)

```

```

        if (comp[i] == -1)
            comp[i] = timer++, calcComp(i);
    for (auto &[u, v] : bridges)
    {
        int cmpU = comp[u];
        int cmpV = comp[v];
        bridgesTree[cmpU].push_back(cmpV);
        bridgesTree[cmpV].push_back(cmpU);
    }
    int mx = -1, node = -1;
    function<void(int, int , int)> dfs2 = [&](int cur, int par, int depth)
    {
        if(depth > mx)
            node = cur, mx = depth;
        for(auto &ch : bridgesTree[cur])
            if(ch != par)
                dfs2(ch, cur, depth + 1);
    };
    dfs2(1, -1, 0);
    mx = -1;
    dfs2(node, -1, 0);
    return mx;
}

```

2.11 Tarjan Articulation Points

```

int tarjan(vector<vector<int>> &graph)
{
    int n = graph.size() - 1;
    vector<int> dfsn(n + 1, -1);
    vector<int> low(n + 1, -1);
    vector<int> isArt(n + 1, 0);
    int timer = 0;
    function<void(int, int)> dfs = [&](int cur, int par) {
        low[cur] = dfsn[cur] = timer++;
        bool rootEdge = false;
        for (auto &ch : graph[cur])
        {
            if (dfsn[ch] == -1)
            {
                dfs(ch, cur);
                low[cur] = min(low[cur], low[ch]);
                if (low[ch] >= dfsn[cur])
                {
                    if(dfsn[cur] == 0 and !rootEdge)
                        rootEdge = true;
                    else
                        isArt[cur] = true;
                }
            }
            else if (ch != par)

```

```

        {
            low[cur] = min(low[cur], dfsn[ch]);
        }
    }
};
dfs(1, -1);
vector<int> artPoints;
for (int i = 1; i <= n; i++)
    if (isArt[i])
        artPoints.push_back(i);
return artPoints.size();
}

```

2.12 Max Flow

```

struct MaxFlow
{
    vector<vector<int>>> adjMat;
    int n;
    vector<int> par;
    MaxFlow(int n) : n(n)
    {
        adjMat = vector<vector<int>>>(n, vector<int>(n, 0));
    }
    void addEdge(int u, int v, int cap)
    {
        adjMat[u][v] += cap;
    }
    int djik(int start, int end, int delta) // Dijkstra Maximum-Minmum Edge
Variant
    {
        par = vector<int>(n, -1);
        priority_queue<pair<int, pii>> pq;
        vector<int> vis(n, 0);
        pq.push({INF, {start, -1}});
        vector<int> dist(n, 0);
        while (pq.size())
        {
            auto [mn, cur] = pq.top();
            pq.pop();
            auto &[i, pr] = cur;
            if (vis[i])
                continue;
            vis[i] = 1;
            dist[i] = mn;
            par[i] = pr;
            if (i == end)
                break;
            for (int j = 0; j < n; j++)
            {
                if (!vis[j] and adjMat[i][j] >= delta)

```

```

        {
            int newMn = min(mn, adjMat[i][j]);
            pq.push({newMn, {j, i}});
        }
    }
    return dist[end];
}
int maxFlow(int start, int end)
{
    int ret = 0;
    int mx = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            mx = max(mx, adjMat[i][j]);
    int delta = 1;
    while (delta <= mx)
        delta <<= 1;
    while (delta)
    {
        while (true)
        {
            int bottleneck = djik(start, end, delta);
            if (bottleneck == 0)
                break;
            int cur = end;
            while (~par[cur])
            {
                addEdge(par[cur], cur, -bottleneck);
                addEdge(cur, par[cur], bottleneck);
                cur = par[cur];
            }
            ret += bottleneck;
        }
        delta >>= 1;
    }
    return ret;
}
};

```

2.13 LCA Mahmoud

```

int n, l;
vector<vector<int>> adj;

int timer;
vector<int> tin, tout;
vector<vector<int>> up;

void dfs(int v, int p)

```

```

{
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i-1]][i-1];

    for (int u : adj[v]) {
        if (u != p)
            dfs(u, v);
    }

    tout[v] = ++timer;
}

bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v)
{
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = l; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}

void preprocess(int root) {
    tin.resize(n);
    tout.resize(n);
    timer = 0;
    l = ceil(log2(n));
    up.assign(n, vector<int>(l + 1));
    dfs(root, root);
}

```

2.14 Dijkstra On Grid

```

int dx[] = {1, -1, 0, 0};
int dy[] = {0, 0, 1, -1};
typedef array<int, 3> entry;
void dijk()
{
    auto valid = [&](int x, int y) -> bool { return (x >= 0 and x < h and y
    >= 0 and y < w); };
    while (pq.size())

```

```

{
    auto [curCost, x, y] = pq.top();
    pq.pop();
    if (dist[x][y] < curCost)
        continue;
    for (int i = 0; i < 4; i++)
    {
        int new_x = x + dx[i], new_y = y + dy[i];
        if (valid(new_x, new_y) and dist[new_x][new_y] > (dist[x][y] +
cost[grid[new_x][new_y] - 'A']))
        {
            dist[new_x][new_y] = dist[x][y] + cost[grid[new_x][new_y] -
'A'];
            pq.push({dist[new_x][new_y], new_x, new_y});
        }
    }
}
}

```

2.15 BiMatching

```

struct BiMatching {
    vector<int> colAssign;
    vector<int> vis;
    vector<vector<int>> adjMat;
    int n, m;

    BiMatching(int n, int m) : n(n), m(m) {
        adjMat = vector<vector<int>>(n, vector<int>(m));
        colAssign = vector<int>(m, -1);
    }

    bool canMatch(int i) {
        for (int j = 0; j < m; j++) {
            if (adjMat[i][j] && !vis[j]) {
                vis[j] = 1;
                if (colAssign[j] == -1 || canMatch(colAssign[j])) {
                    colAssign[j] = i;
                    return true;
                }
            }
        }
        return false;
    }

    void addEdge(int u, int v) {
        adjMat[u][v] = 1;
    }

    int maxMatching() {
        int maxFlow = 0;
    }
}

```

```

        for (int i = 0; i < n; i++) {
            vis = vector<int>(m, 0);
            if (canMatch(i))
                maxFlow++;
        }
        return maxFlow;
    }
};

```

2.16 HLD

```

struct HLD
{
    int n, timer;
    vector<int> flat, sz, tp, dep, par;
    vector<vector<int>> graph;
    FenwickTree bit;
    HLD(vector<vector<int>> &graph) : graph(graph), n(graph.size() - 1)
    {
        timer = 0;
        sz = flat = tp = dep = par = vector<int>(n + 1, 0);
        preDfs(1);
        dfs(1);
        bit.resize(n + 1);
        for(int i = 1; i <= n; i++)
            update(i, 1);
    }
    void preDfs(int cur)
    {
        sz[cur] = 1;
        for (auto &ch : graph[cur])
        {
            graph[ch].erase(find(all(graph[ch]), cur));
            dep[ch] = dep[cur] + 1;
            par[ch] = cur;
            preDfs(ch);
            sz[cur] += sz[ch];
        }
        auto __cmp = [&](int u, int v) { return (sz[u] < sz[v]); };
        int heavy = max_element(all(graph[cur]), __cmp) -
graph[cur].begin();
        if (graph[cur].size())
        {
            swap(graph[cur][0], graph[cur][heavy]);
        }
    }
    void dfs(int cur, int top = 1)
    {
        flat[cur] = ++timer, tp[cur] = top;
        for (int i = 0; i < graph[cur].size(); i++)
        {

```

```

        auto &ch = graph[cur][i];
        dfs(ch, !i ? top : ch);
    }
}
void update(int cur, int val)
{
    bit.update(flat[cur], val);
}
int query(int u, int v)
{
    int ret = 0;
    for (; tp[u] != tp[v]; u = par[tp[u]])
    {
        if (dep[tp[u]] < dep[tp[v]])
            swap(u, v);
        ret += bit.get(flat[tp[u]], flat[u]);
    }
    if (dep[u] < dep[v])
        swap(u, v);
    ret += bit.get(flat[v], flat[u]);
    return ret;
}
};

```

2.17 SACK

```

int c[N];
int n;
vector<int> graph[N];
int heavy[N];
int freq[N];
int subtree[N];
int answer = 0;
int ans[N];
void update(int val, int delta)
{
    answer -= (freq[val] > 0);
    freq[val] += delta;
    answer += (freq[val] > 0);
}
void compress()
{
    map<int, int> mp;
    for (int i = 1; i <= n; i++)
        if (!mp.count(c[i]))
            mp[c[i]] = mp.size() + 1;
    for (int i = 1; i <= n; i++)
        c[i] = mp[c[i]];
}
void dfs0(int cur, int par)
{

```

```

    for (auto &ch : graph[cur])
    {
        if (ch == par)
            continue;
        dfs0(ch, cur);
        if (subtree[heavy[cur]] < subtree[ch])
            heavy[cur] = ch;
        subtree[cur] += subtree[ch];
    }
    subtree[cur]++;
}
void calc(int cur, int par, int delta)
{
    update(c[cur], delta);
    for (auto &ch : graph[cur])
        if (ch != par)
            calc(ch, cur, delta);
}
void dfs(int cur, int par, int keep)
{
    for (auto &ch : graph[cur])
    {
        if (ch == par || ch == heavy[cur])
            continue;
        dfs(ch, cur, 0);
    }
    if (heavy[cur])
        dfs(heavy[cur], cur, 1);
    for (auto &ch : graph[cur])
    {
        if (ch == par || ch == heavy[cur])
            continue;
        calc(ch, cur, 1);
    }
    update(c[cur], 1);
    ans[cur] = answer;
    if (!keep)
        calc(cur, par, -1);
}

```

3. DP

3.1 LIS

```

int n, mx = 0;
int arr[N] = {};
vector<int> dp(N, INF);
void solve()
{
    // memset(dp, -1, sizeof dp);
}

```

```

    cin >> n;
    for (size_t i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    dp[0] = -INF;
    for (int i = 0; i < n; i++)
    {
        int l = upper_bound(dp.begin(), dp.end(), arr[i]) - dp.begin();
        if (dp[l - 1] < arr[i] && arr[i] < dp[l])
            dp[l] = arr[i];
    }
    for (int l = 0; l <= n; l++)
    {
        if (dp[l] < INF)
            mx = l;
    }
    cout << mx << endl;
}

```

3.2 LIS With Data Structures

```

void solve()
{
    int n, k;
    cin >> n >> k;
    vector<int> v(n);
    int mx = 0;
    for (auto &x : v)
        cin >> x, mx = max(mx, x);
    vector<BIT> dp(k + 1, BIT(mx + 1));

    for (int i = 0; i < n; i++)
    {
        dp[1].update(v[i] + 1, 1);
        for (int j = 1; j < k; j++)
        {
            int add = dp[j].pref(v[i]);
            dp[j + 1].update(v[i] + 1, add);
        }
    }
    cout << dp[k].pref(mx + 1);
}

```

3.3 Digits

```

void solve()
{

```

```

string l, r;
int k;
cin >> l >> r >> k;
if(r.size() > l.size())
    l = string(r.size() - l.size(), '0') + l;
k = min(k, 99ll);
int dp[r.size()][2][2][100][100];
int n = r.size();
memset(dp, -1, sizeof(dp));
function<int(int, int, int, int, int)> sol = [&](int i, int
canExceed, int canDown, int remNum, int remSum)->int
{
    if(i == n)
        return !(remSum or remNum);
    auto &ret = dp[i][canExceed][canDown][remNum][remSum];
    if(~ret)
        return ret;
    ret = 0;
    for(int d = 0; d < 10; d++)
    {
        if(!canExceed and d > (r[i] - '0'))
            continue;
        if(!canDown and d < (l[i] - '0'))
            continue;
        ret += sol(i + 1, canExceed | (d != r[i] - '0'), canDown | (d
!= l[i] - '0'), (remNum + (d * (int)powl(10, n - i - 1))) % k, (remSum + d
% k);
    }
    return ret;
};
cout << sol(0, 0, 0, 0, 0) << endl;
}

```

4. Math

4.1 Matrix Exponentiation

```

const int mod = 1e9 + 7;
using Row = vector<long long>;
using Matrix = vector<Row>;

Matrix mul(Matrix &a, Matrix &b) {
    int n = a.size(), m = a[0].size(), k = b[0].size();
    Matrix res(n, Row(k));
    for(int i = 0; i < n; ++i)
        for(int j = 0; j < k; ++j)
            for(int o = 0; o < m; ++o) {
                res[i][j] += 1ll * a[i][o] * b[o][j] % mod;
                if(res[i][j] >= mod) res[i][j] -= mod;
            }
    return res;
}

```

```

        if(res[i][j] < 0) res[i][j] += mod;
    }
    return res;
}

Matrix power(Matrix a, long long b) {
    int n = a.size();
    Matrix res(n, Row(n));
    for(int i = 0; i < n; ++i) res[i][i] = 1;

    while(b) {
        if(b&1) res = mul(res, a);
        a = mul(a, a), b >>= 1;
    }

    return res;
}

void destroy(){

    long long n;
    cin >> n;

    Matrix S = {{0,0,0,0,0,1}};
    Matrix T(6,Row(6)); // 6 * 6

    for (int i = 0; i < 6;i++){
        T[i][5] = 1;
    }
    for (int j = 0; j < 5;j++){ // excluded last col cuz I already setted
it        T[j + 1][j] = 1; // all other cells equal zero
    }
    T = power(T, n);
    S = mul(S, T);
    cout << S[0][5] << endl;
}

```

4.2 Big Int Mul

```

string mul_two_big_int(const string &s1, const string &s2) {
    int n = s1.size(), m = s2.size();

    vector<int> poly1(n), poly2(m);
    for (int i = 0; i < n; ++i) {
        poly1[n-i-1] = s1[i] - '0';
    }

    for (int i = 0; i < m; ++i) {
        poly2[m-i-1] = s2[i] - '0';
    }
}

```

```

vector<int> ans = multiply(poly1, poly2);
int k = ans.size();

for (int i = 0; i < k - 1; ++i) {
    ans[i + 1] += ans[i] / 10;
    ans[i] = ans[i] % 10;
}

string final = to_string(ans[k - 1]);
for (int i = k - 2; i >= 0; --i) {
    final += (char)(ans[i] + '0');
}

for (int i = 0; i < k; ++i) {
    if(final[i] != '0') return final.substr(i);
}
return "0";
}

string power_of_big_int(string s, int p) {
    string ans = "1";
    while (p) {
        if(p&1) ans = mul_two_big_int(ans, s);
        s = mul_two_big_int(s, s);
        p >>= 1;
    }
    return ans;
}

```

4.3 Exact Match but multi with num chars

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
    }
}

```

```

        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + (int)b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

void solve(int tc) {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<int> poly1(n), poly2(m);

    vector<int> ans_match(n);

    for (int i = 0; i < 26; ++i) {
        for (int j = 0; j < n; ++j) {
            poly1[j] = (s[j] - 'a') == i;
        }
        for (int j = 0; j < m; ++j) {
            poly2[j] = (patt[m-j-1] - 'a') == i;
        }
        vector<int> ans = multiply(poly1, poly2);
    }
}

```

```

        for (int j = 0; j < n; ++j) {
            ans_match[j] += ans[m-1+j];
        }
    }

    int tot = 0;
    vector<int> pos;
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');
    for (int i = 0; i < n; ++i) {
        if(ans_match[i] == m - wild_cnt) {
            ++tot;
            pos.push_back(i);
        }
    }

    cout << tot << "\n";
    for(auto & p : pos) cout << p << " ";
    cout << "\n";

}

```

4.4 FFT

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }
}

```

```

    }
}

if (invert) {
    for (cd & x : a)
        x /= n;
}
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() * m)
        n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

```

4.5 FFT Mod

```

#define rep(aa, bb, cc) for(int aa = bb; aa < cc; aa++)
#define sz(a) (int)a.size()
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i, k, 2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
            // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
        }
}
/// include-line

```

```

        auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
    /// exclude-line
        C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
    /// exclude-line
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
    }
}

template<int M> vi convMod(const vi &a, const vi &b) {
    if (a.empty() || b.empty()) return {};
    vi res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}

```

4.6 FWHD

```

int add(int a, int b) {
    return (a + b) % mod;
}

int sub(int a, int b) {
    return (a - b + mod) % mod;
}

ll poww(ll a, ll b) {
    ll ret = 1;
    while (b) {
        if (b & 1) ret = ret * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return ret;
}

```

```

void fwht(vector<int> &a, int inv, int f) {
    int sz = a.size();
    for (int len = 1; 2 * len <= sz; len <<= 1) {
        for (int i = 0; i < sz; i += 2 * len) {
            for (int j = 0; j < len; j++) {
                int x = a[i + j];
                int y = a[i + j + len];

                if (f == 0) {
                    if (!inv) a[i + j] = y, a[i + j + len] = add(x, y);
                    else a[i + j] = sub(y, x), a[i + j + len] = x;
                }
                else if (f == 1) {
                    if (!inv) a[i + j + len] = add(x, y);
                    else a[i + j + len] = sub(y, x);
                }
                else {
                    a[i + j] = add(x, y);
                    a[i + j + len] = sub(x, y);
                }
            }
        }
    }
}

vector<int> mul(vector<int> a, vector<int> b, int f) { // 0:AND, 1:OR,
2:XOR
    int sz = a.size();
    fwht(a, 0, f); fwht(b, 0, f);
    vector<int> c(sz);
    for (int i = 0; i < sz; ++i) {
        c[i] = 1ll * a[i] * b[i] % mod;
    }
    fwht(c, 1, f);
    if (f) {
        int sz_inv = poww(sz, mod - 2);
        for (int i = 0; i < sz; ++i) {
            c[i] = 1ll * c[i] * sz_inv % mod;
        }
    }
    return c;
}

```

4.7 NTT

```

const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.

ll modpow(ll b, ll e) {

```

```

    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}

// Primitive Root of the mod of form 2^a * b + 1
int generator () {
    vector<int> fact;
    int phi = mod-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=mod; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= modpow (res, phi / fact[i]) != 1;
        if (ok) return res;
    }
    return -1;
}

int modpow(int b, int e, int m) {
    int ans = 1;
    for (; e; b = (ll)b * b % m, e /= 2)
        if (e & 1) ans = (ll)ans * b % m;
    return ans;
}

void ntt(vector<int> &a) {
    int n = (int)a.size(), L = 31 - __builtin_clz(n);
    vector<int> rt(2, 1); // erase the static if you want to use two
moduli;
    for (int k = 2, s = 2; k < n; k *= 2, s++) { // erase the static if you
want to use two moduli;
        rt.resize(n);
        int z[] = {1, modpow(root, mod >> s, mod)};
        for (int i = k; i < 2*k; ++i) rt[i] = (ll)rt[i / 2] * z[i & 1] %
mod;
    }
    vector<int> rev(n);
    for (int i = 0; i < n; ++i) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; ++i) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; ++j) {
                int z = (ll)rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
                a[i + j + k] = ai - z + (z > ai ? mod : 0);
                ai += (ai + z >= mod ? z - mod : z);
            }
        }
    }
}

```

```

    }
    }
}

vector<int> conv(const vector<int> &a, const vector<int> &b) {
    if (a.empty() || b.empty()) return {};
    int s = (int)a.size() + (int)b.size() - 1, B = 32 - __builtin_clz(s), n
= 1 << B;
    int inv = modpow(n, mod - 2, mod);
    vector<int> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    for (int i = 0; i < n; ++i) out[-i & (n - 1)] = (ll)L[i] * R[i] % mod *
inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

ll CRT(ll a, ll m1, ll b, ll m2) {
    __int128 m = m1*m2;
    ll ans = a*m2%m*modpow(m2, m1-2, m1)%m + m1*b%m*modpow(m1, m2-2, m2)%m;
    return ans % m;
}

/*

int mod, root, desired_mod = 1000000007;
const int mod1 = 167772161;
const int mod2 = 469762049;
const int mod3 = 754974721;
const int root1 = 3;
const int root2 = 3;
const int root3 = 11;

int CRT(int a, int b, int c, int m1, int m2, int m3) {
    __int128 M = (__int128)m1*m2*m3;
    ll M1 = (ll)m2*m3;
    ll M2 = (ll)m1*m3;
    ll M3 = (ll)m2*m1;

    int M_1 = modpow(M1%m1, m1 - 2, m1);
    int M_2 = modpow(M2%m2, m2 - 2, m2);
    int M_3 = modpow(M3%m3, m3 - 2, m3);

    __int128 ans = (__int128)a*M1*M_1;
    ans += (__int128)b*M2*M_2;
    ans += (__int128)c*M3*M_3;

    return (ans % M) % desired_mod;
}

*/

```

4.8 Pow

```
vector<int> poly_pow(vector<int> poly, int p, int limit = 1e9) {
    vector<int> ans{1};
    while (p) {
        if(p&1) ans = conv(ans, poly);
        poly = conv(poly, poly);
        ans.resize(limit + 1);
        poly.resize(limit + 1);
        p >>= 1;
    }
    return ans;
}
```

4.9 String Matching With Wild Cards

```
using cd = complex<double>;
const double PI = acos(-1), eps = 5e-4; // If you get a wrong answer you
can change the eps lower or higher till you pass

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
```

```

        x /= n;
    }
}

vector<cd> multiply(vector<cd> const& a, vector<cd> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + (int)b.size())
        n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    return fa;
}

void solve(int tc) {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<cd> poly1(n), poly2(m);

    for (int i = 0; i < n; ++i) {
        double angle = 2*PI*(s[i]-'a')/26;
        poly1[i] = cd(cos(angle), sin(angle));
    }
    for (int i = 0; i < m; ++i) {
        if(patt[m-i-1] == '*') poly2[i] = cd(0,0); // Wild Card
        else {
            double angle = 2*PI*(patt[m-i-1]-'a')/26;
            poly2[i] = cd(cos(angle), -sin(angle));
        }
    }

    vector<cd> ans = multiply(poly1, poly2);
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');

    int tot = 0;
    vector<int> pos;
    for (int i = 0; i < n; ++i) {
        if(fabs(ans[m-1+i].real() - (m - wild_cnt)) < eps && fabs(ans[m-1+i].imag()) < eps) {
            ++tot;
            pos.push_back(i);
        }
    }

    cout << tot << "\n";
}

```

```

    for(auto & p : pos) cout << p << " ";
    cout << "\n";

}

```

5. Geometry

5.1 Geometry

```

typedef ld T;
typedef complex<T> point;
#define real real()
#define imag imag()

T sq(point p) {
    return p.real * p.real + p.imag * p.imag;
}
T abs(point p)
{
    return sqrt(p.real * p.real + p.imag * p.imag);
}
int sgn(T val)
{
    if (val > EPS) return 1;
    if (val < -EPS) return -1;
    else return 0;
}

// Transformations
point translate(point a, point b)
{
    return a + b;
}
point scale(point c, T factor, point b)
{
    return c + (b - c) * factor;
}
point rotate(point a, point from, T theta)
{
    // theta in radian
    point v = a - from;
    point rot = { cos(theta) , sin(theta) };
    return from + rot * v;
}
// i want know image of r from transformation of p , q
point linearTransfo(point p, point q, point r, point fp, point fq) {
    return fp + (r - p) * (fq - fp) / (q - p);
}

// dot & cross

```

```

T dot(point a, point b)
{
    return a.real * b.real + a.imag * b.imag;
}

T angleBetweenVectors(point v, point w) {
    // small angle
    T cosTheta = dot(v, w) / abs(v) / abs(w);
    return acos(max(-1 * (T)1.0, min((T)1.0, cosTheta)));
}

bool isPerpendicular(point v , point w)
{
    return fabs(dot(v , w)) < EPS;
}

point getLeftPerp(point v)
{
    return { -v.imag , v.real };
}

T cross(point v, point w)
{
    return v.real * w.imag - v.imag * w.real;
}

T orient(point a, point b, point c)
{
    // know point c in right or left
    // from a to b
    // if oriented == 0 then a , b, c in same line
    return cross(b - a, c - a);
}

T orientedAngle(point a, point b, point c)
{
    // get angle between a-b and a-c if large or small
    ld angle = angleBetweenVectors(b - a, c - a);
    if (orient(a, b, c) > 0) return angle;
    else return (ld)360 - angle;
}

bool inAngle(point a, point b, point c, point y)
{
    //check if point y in angle c-a-b in counterclockwise
    return orient(a, b, y) >= 0 and orient(a, c, y) <= 0;
}

//////////////////// Lines
////////////////////
struct line {
    point v; T c;

    line(point v, T c) : v(v), c(c) {}

    // from equation ax+by = c
    line(T a, T b, T _c) {
        v = { b, -a };

```

```

        c = -c;
    }

    //line from two points
    line(point p, point q) {
        v = q - p;
        c = cross(v, p);
    }
    // return side of point from line
    T side(point p) { return cross(v, p) - c; }
    //return distance between p and line
    ld dist(point p) { return abs(side(p)) / abs(v); }
    // distance square
    T sqDist(point p) { return side(p) * side(p) / (T)sq(v); }
    //get line perpendicular about line
    line prepThrought(point p) { return { p, p + getLeftPerp(p) }; }

    bool cmpProj(point p, point q) {
        return dot(v, p) < dot(v, q);
    }
    //translate line by vector t
    line translate(point t) { return { v, c + cross(v,t) }; }
    //translate line by distance dist
    line shiftLeft(T dist) { return { v, c + dist * abs(v) }; }
    // projection of point on line
    point proj(point p) { return p - getLeftPerp(v) * side(p) / sq(v); }
    // reflection of point about line
    point refl(point p) { return p - getLeftPerp(v) * (T)2.0 * side(p) /
sq(v); }
};

// return intersection point between two lines
bool inter(line l1, line l2, point& out) {
    T d = cross(l1.v, l2.v);
    if (fabs(d) < EPS) return false;
    out = (l2.v * l1.c - l1.v * l2.c) / d; // requires floating-point
coordinates
    return true;
}

// return line that divide two lines
// if interior == 1 => enter
line bisector(line l1, line l2, bool interior) {
    assert(cross(l1.v, l2.v) != 0); // l1 and l2 cannot be parallel!
    T sign = interior ? 1 : -1;
    return { l2.v / abs(l2.v) + l1.v / abs(l1.v) * sign,
        l2.c / abs(l2.v) + l1.c / abs(l1.v) * sign };
}

//////////
segments//////////
bool inDisk(point a, point b, point p) {
    return dot(a-p, b-p) <= EPS;
}

// check if c in segment of line (st-end)
bool onSegment(point a, point b, point c){
    return orient(a, b, c) == 0 && inDisk(a, b, c);
}

```

```

}
// if exist intersection between two segments
bool properInter(point a, point b, point c, point d, point &out) {
    T oa = orient(c,d,a),
      ob = orient(c,d,b),
      oc = orient(a,b,c),
      od = orient(a,b,d);
// Proper intersection exists iff opposite signs
    if (sgn(oa)*sgn(ob) < 0 && sgn(oc)*sgn(od) < 0) {
        out = (a*ob - b*oa) / (ob-oa);
        return true;
    }
    return false;
}
// get all intersections points in segments
set<pair<ld,ld>> allSegmentsInters(point a, point b, point c, point d) {
    set<pair<ld,ld>> s;
    point out;
    if(a == c || a == d){
        s.insert(make_pair(a.real, a.imag));
    }
    if(b == c || b == d){
        s.insert(make_pair(b.real, b.imag));
    }
    if(s.size()) return s;

    if (properInter(a,b,c,d,out)) return {make_pair(out.real, out.imag)};
    if (onSegment(c,d,a)) s.insert(make_pair(a.real, a.imag));
    if (onSegment(c,d,b)) s.insert(make_pair(b.real, b.imag));
    if (onSegment(a,b,c)) s.insert(make_pair(c.real, c.imag));
    if (onSegment(a,b,d)) s.insert(make_pair(d.real, d.imag));

    return s;
}
// get neareast distance between point and seg
T distanceSegPoint(point a, point b, point p) {
    if (a != b) {
        line l(a,b);
        if (l.cmpProj(a,p) && l.cmpProj(p,b)) // if closest to projection
            return l.dist(p); // output distance to line
    }
    return min(abs(p-a), abs(p-b)); // otherwise distance to A or B
}
// get neareast distance between seg and seg
T distanceSegSeg(point a, point b, point c, point d) {
    point dummy;
    if (properInter(a,b,c,d,dummy))
        return 0;
    return min({distanceSegPoint(a,b,c), distanceSegPoint(a,b,d),
                distanceSegPoint(c,d,a), distanceSegPoint(c,d,b)});
}

//////////////////// Polygons

```

```

////////////////////////////////////
T areaTriangle(point a, point b, point c) {
    return abs(cross(b-a, c-a)) / 2.0;
}

bool above(point a, point p) {
    return p.imag >= a.imag;
}

bool crossesRay(point a, point p, point q) {
    return (above(a,q)- above(a,p)) * orient(a,p,q) > 0;
}

T areaPolygon(vector<point> p) {
    ld area = 0.0;
    for (int i = 0, n = p.size(); i < n; i++) {
        area += cross(p[i], p[(i+1)%n]); // wrap back to 0 if i == n - 1
    }
    //if area is + => counterClockwise
    return abs(area) / 2.0;
}

// Returns the distance from point p to a ray defined by origin a and
// direction (b - a)
T distancePointRay(point p, point a, point b) {
    point dir = b - a;          // Ray direction vector
    point v = p - a;           // Vector from ray origin to point

    T u = dot(v, dir);          // Projection scalar

    if (u <= 0) {                // Point is behind the ray origin
        return abs(v);           // Distance from p to a
    }
    else {                       // Point is in front of the ray
        // Compute closest point on the ray
        point closest = a + (u / norm(dir)) * dir;
        return abs(p - closest); // Distance from p to closest point
    }
}

//////////////////////////////////// Circles
////////////////////////////////////

pair<point, T> circumCircle(point a, point b, point c) {
    b = b-a, c = c-a; // consider coordinates relative to A
    assert(cross(b,c) != 0); // no circumcircle if A,B,C aligned
    return {a + getLeftPerp(b*sq(c) - c*sq(b))/cross(b,c)/(T)2,
abs(getLeftPerp(b*sq(c) - c*sq(b))/cross(b,c)/(T)2)};
}

// return 1 if exist intersection points between line & circle ( to => out
// pair) else return 0
int circleLine(point o, double r, line l, pair<point,point> &out) {
    double h2 = r*r - l.sqDist(o);
    if (h2 >= 0) { // the line touches the circle
        point p = l.proj(o); // point P
        point h = l.v* (T)(sqrt(h2)/abs(l.v)); // vector parallel to l, of
length h

```

```

        out = {p-h, p+h};
    }
    return 1 + sgn(h2);
}

// return 1 if exist intersection points between circle & circle ( to =>
// out pair) else return 0
int circleCircle(point o1, T r1, point o2, T r2, pair<point,point> &out) {
    point d=o2-o1; T d2=sq(d);
    if (d2 == 0) {assert(r1 != r2); return 0;} // concentric circles
    T pd = (d2 + r1*r1 - r2*r2)/2; // = |O_1P| * d
    T h2 = r1*r1 - pd*pd/d2; // = h^2
    if (h2 >= 0) {
        point p = o1 + d*pd/d2, h = getLeftPerp(d)*sqrt(h2/d2);
        out = {p-h, p+h};
    }
    return 1 + sgn(h2);
}

// get tangent points between two circles inner or outer
int tangents(point o1, T r1, point o2, T r2, bool inner,
vector<pair<point,point>> &out) {
    if (inner) r2 = -r2;
    point d = o2-o1;
    T dr = r1-r2, d2 = sq(d), h2 = d2-dr*dr;
    if (d2 == 0 || h2 < 0) {assert(h2 != 0); return 0;}
    for (T sign : {-1,1}) {
        point v = (d*dr + getLeftPerp(d)*sqrt(h2)*sign)/d2;
        out.push_back({o1 + v*r1, o2 + v*r2});
    }
    return 1 + (h2 > 0);
}

```

6. Data Structures

6.1 Segment Tree

```

struct SegTree
{
#define left (ni << 1) | 1
#define right ((ni + 1) << 1)
#define mid (lx + rx) >> 1
    struct Node
    {
        int sum;
        Node()
        {
            sum = 0;
        }
        Node(int x)
        {

```

```

        sum = x;
    }
    void change(int x)
    {
        sum = x;
    }
};
const Node netural = Node();
vector<Node> tree;
int treeSize;
SegTree(int n)
{
    treeSize = 1;
    while (treeSize < n)
        treeSize *= 2;
    tree.assign(2 * treeSize, netural);
}
Node merge(Node &lf, Node &rf)
{
    return Node(lf.sum + rf.sum);
}
void build(vector<int> &arr, int ni, int lx, int rx)
{
    if (rx - lx == 1)
    {
        if (lx < arr.size())
            tree[ni].change(arr[lx]);
        return;
    }
    build(arr, left, lx, mid);
    build(arr, right, mid, rx);
    tree[ni] = merge(tree[left], tree[right]);
}
void build(vector<int> &arr)
{
    build(arr, 0, 0, treeSize);
}
void update(int idx, int x, int ni, int lx, int rx)
{
    if (rx - lx == 1)
        return void(tree[ni].change(x));
    if (idx < mid)
        update(idx, x, left, lx, mid);
    else
        update(idx, x, right, mid, rx);
    tree[ni] = merge(tree[left], tree[right]);
}
void update(int idx, int x)
{
    update(idx, x, 0, 0, treeSize);
}
Node get(int l, int r, int ni, int lx, int rx)
{
    if (lx >= l and rx <= r)

```

```

        return tree[ni];
    if (lx >= r or rx <= l)
        return netural;
    Node lf = get(l, r, left, lx, mid);
    Node rf = get(l, r, right, mid, rx);
    return merge(lf, rf);
}
int get(int l, int r)
{
    return get(l, r, 0, 0, treeSize).sum;
}
#undef right
#undef left
#undef mid
};

```

6.2 Lazy Segment Tree

```

struct SegTree
{
#define left (ni << 1) | 1
#define right ((ni + 1) << 1)
#define mid (lx + rx) >> 1
    struct Node
    {
        int sum;
        int lazy;
        bool isLazy;
        Node()
        {
            sum = 0;
            lazy = 0;
            isLazy = false;
        }
        Node(int x)
        {
            sum = x;
            lazy = 0;
            isLazy = false;
        }
        void change(int x, int lx, int rx)
        {
            sum += (rx - lx) * x;
            lazy += x;
            isLazy = true;
        }
    };
    int treeSize;
    vector<Node> tree;
    const Node neutral = Node();
    SegTree(int n)

```

```

{
    treeSize = 1;
    while (treeSize < n)
        treeSize <= 1;
    tree.assign(treeSize << 1, neutral);
}
Node merge(Node &lf, Node &rf)
{
    return Node(lf.sum + rf.sum);
}
void propagate(int ni, int lx, int rx)
{
    if (!(tree[ni].isLazy) || rx - lx == 1)
        return;
    tree[left].change(tree[ni].lazy, lx, mid);
    tree[right].change(tree[ni].lazy, mid, rx);
    tree[ni].lazy = 0;
    tree[ni].isLazy = false;
}
void build(vector<int> &arr, int ni, int lx, int rx)
{
    if (rx - lx == 1)
    {
        if (lx < arr.size())
            tree[ni].change(arr[lx], lx, rx);
        return;
    }
    build(arr, left, lx, mid);
    build(arr, right, mid, rx);
    tree[ni] = merge(tree[left], tree[right]);
}
void update(int l, int r, int v, int ni, int lx, int rx)
{
    propagate(ni, lx, rx);
    if (lx >= l and rx <= r)
    {
        tree[ni].change(v, lx, rx);
        return;
    }
    if (lx >= r or rx <= l)
        return;
    update(l, r, v, left, lx, mid);
    update(l, r, v, right, mid, rx);
    tree[ni] = merge(tree[left], tree[right]);
}
void update(int l, int r, int v)
{
    update(l, r, v, 0, 0, treeSize);
}
void update(int idx, int v)
{
    update(idx, idx + 1, v);
}
Node get(int l, int r, int ni, int lx, int rx)

```

```

{
    propagate(ni, lx, rx);
    if (lx >= l and rx <= r)
        return tree[ni];
    if (lx >= r or rx <= l)
        return neutral;
    Node lf = get(l, r, left, lx, mid);
    Node rf = get(l, r, right, mid, rx);
    return merge(lf, rf);
}
int get(int l, int r)
{
    return get(l, r, 0, 0, treeSize).sum;
}
int get(int idx)
{
    return get(idx, idx + 1);
}
#undef left
#undef right
#undef mid
};

```

6.3 BIT

```

struct BIT
{
    vector<int> tree;
    int treeSize;
    BIT(int n) : treeSize(n)
    {
        tree.assign(n + 1, 0);
    }
    void update(int idx, int delta)
    {
        while (idx <= treeSize)
        {
            tree[idx] += delta;
            idx += (idx & -idx);
        }
    }
    int pref(int idx)
    {
        int ret = 0;
        while (idx > 0)
        {
            ret += tree[idx];
            idx -= (idx & -idx);
        }
        return ret;
    }
}

```

```

    int get(int l, int r)
    {
        return pref(r) - pref(l - 1);
    }
};

```

6.4 Range Update BIT

```

struct BIT
{
    vector<int> tree;
    vector<int> tree2;
    int treeSize;
    BIT(int n) : treeSize(n)
    {
        tree.assign(n + 1, 0);
        tree2.assign(n + 1, 0);
    }
    void add(int idx, int delta, vector<int> &tree)
    {
        while (idx <= treeSize)
        {
            tree[idx] += delta;
            idx += (idx & -idx);
        }
    }
    int query(int idx, vector<int> &tree)
    {
        int ret = 0;
        while (idx > 0)
        {
            ret += tree[idx];
            idx -= (idx & -idx);
        }
        return ret;
    }
    void update(int l, int r, int delta)
    {
        add(l, delta, tree);
        add(r + 1, -delta, tree);
        add(l, delta * (l - 1), tree2);
        add(r + 1, -delta * r, tree2);
    }
    int pref(int idx)
    {
        return query(idx, tree) * idx - query(idx, tree2);
    }
    int get(int l, int r)
    {
        return pref(r) - pref(l - 1);
    }
};

```

```
    }
};
```

6.5 Sqrt Decomposition

```
const int sqt = 450;
int n{0}, q{0};
int lazy[sqt + 1]{};
int lazySet[sqt + 1]{};
int buckets[sqt + 1]{};
int arr[N]{};
void build(int buc_num)
{
    buckets[buc_num] = 0;
    for (int i = buc_num * sqt; i < min(n, (buc_num + 1) * sqt); i++)
    {
        if (lazySet[buc_num])
            arr[i] = lazySet[buc_num];
        if (lazy[buc_num])
            arr[i] += lazy[buc_num];
        buckets[buc_num] += arr[i];
    }
    lazySet[buc_num] = lazy[buc_num] = 0;
}
void updateSet(int l, int r, int x)
{
    build(l / sqt);
    build(r / sqt);
    for (int i = l; i <= r;)
    {
        if (i % sqt == 0 and i + sqt - 1 <= r)
        {
            buckets[i / sqt] = sqt * x;
            lazy[i / sqt] = 0;
            lazySet[i / sqt] = x;
            i += sqt;
        }
        else
        {
            buckets[i / sqt] -= arr[i];
            arr[i] = x;
            buckets[i / sqt] += arr[i];
            i++;
        }
    }
};
void update(int l, int r, int x)
{
    build(l / sqt);
    build(r / sqt);
```

```

    for (int i = l; i <= r;)
    {
        if (i % sqrt == 0 and i + sqrt - 1 <= r)
        {
            buckets[i / sqrt] += sqrt * x;
            lazy[i / sqrt] += x;
            i += sqrt;
        }
        else
        {
            buckets[i / sqrt] -= arr[i];
            arr[i] += x;
            buckets[i / sqrt] += arr[i];
            i++;
        }
    }
};
int query(int l, int r)
{
    build(l / sqrt);
    build(r / sqrt);
    int ret = 0;
    for (int i = l; i <= r;)
    {
        if (i % sqrt == 0 and i + sqrt - 1 <= r)
        {
            ret += buckets[i / sqrt];
            i += sqrt;
        }
        else
        {
            ret += arr[i];
            i++;
        }
    }
    return ret;
}

```

6.6 Mo's Algorithm

```

const int sqrt = 320;
struct query
{
    int l, r, i;
    bool operator<(query &other)
    {
        int b1 = l / sqrt;
        int b2 = other.l / sqrt;
        if(b1 != b2) return b1 < b2;
        return r < other.r;
    }
}

```

```

};

int freq[N]{};
int n, q;
int answer = 0;
int arr[N]{};
void add(int idx)
{
    int val = arr[idx];
    if(val > N)
        return;
    answer -= (freq[val] == val);
    freq[val]++;
    answer += (freq[val] == val);
}
void rem(int idx)
{
    int val = arr[idx];
    if(val > N)
        return;

    answer -= (freq[val] == val);
    freq[val]--;
    answer += (freq[val] == val);
}

void MosAlgo(vector<query> &queries, vector<int> &ans)
{
    sort(all(queries));
    int l = 0, r = -1;
    for(auto &[L, R, idx] : queries)
    {
        while (r < R) add(++r);
        while(l > L) add(--l);
        while(r > R) rem(r--);
        while(l < L) rem(l++);
        ans[idx] = answer;
    }
}

```

6.7 Segment Tree Mahmoud

```

struct Node{
    long long val;
    Node() {}
    Node(long long vall) {
        val = vall;
    }
};

struct SegTree{

```

```
Node query(int node, int left, int right, int ql, int qr){
    if (left >= ql && right <= qr) return tree[node];
```

[illegible]

6.8 Segment Tree Pointers Mahmoud

```

struct Vertex {
    int left, right;
    int sum = 0;
    Vertex *left_child = nullptr, *right_child = nullptr;

    Vertex(int lb, int rb) {
        left = lb;
        right = rb;
    }

    void extend() {
        if (!left_child && left + 1 < right) {
            int t = (left + right) / 2;
            left_child = new Vertex(left, t);
            right_child = new Vertex(t, right);
        }
    }

    void add(int k, int x) {
        extend();
        sum += x;
        if (left_child) {
            if (k < left_child->right)
                left_child->add(k, x);
            else
                right_child->add(k, x);
        }
    }

    int get_sum(int lq, int rq) {
        if (lq <= left && right <= rq)
            return sum;
    }
}

```

```

        if (max(left, lq) >= min(right, rq))
            return 0;
        extend();
        return left_child->get_sum(lq, rq) + right_child->get_sum(lq, rq);
    }
};

```

6.9 Segment Tree Pointers Kero

```

struct Node
{
    int sum;
    // int lx, rx; // lx (inclusive), rx(exclusive)
    Node *l, *r;
    Node() : sum(0), l(NULL), r(NULL)
    {
    }
    Node(int sum) : sum(sum), l(NULL), r(NULL)
    {
    }
};

const int R = 1e9 + 1;
int mul(int a, int b)
{
    return (a * b) % mod;
}

void merge(Node *ret)
{
    ret->sum = 1;
    ret->sum = mul(ret->sum, ret->l ? ret->l->sum : 0);
    ret->sum = mul(ret->sum, ret->r ? ret->r->sum : 0);
}

Node *merge(Node *lf, Node *rf)
{
    Node *ret = NULL;
    if (!lf and !rf)
        return NULL;
    ret = new Node();
    ret->sum = 1;
    if (lf)
    {
        ret->sum = mul(ret->sum, lf->sum);
        delete lf;
        lf = NULL;
    }
    if (rf)
    {
        ret->sum = mul(ret->sum, rf->sum);
        delete rf;
        rf = NULL;
    }
}

```

```
        return ret;
    }
    Node *root;
    void upd(int idx, int delta, Node *cur, int lx, int rx)
    {
        if (rx - lx == 1)
        {
            cur->sum += delta;
            return;
        }
        int mid = (lx + rx) / 2;
        if (idx < mid)
        {
            if (!cur->l)
                cur->l = new Node();
            upd(idx, delta, cur->l, lx, mid);
        }
        else
        {
            if (!cur->r)
                cur->r = new Node();
            upd(idx, delta, cur->r, mid, rx);
        }
        merge(cur);
    }
    void upd(int idx, int delta)
    {
        upd(idx, delta, root, 0, R);
    }
    Node *get(int l, int r, Node *cur, int lx, int rx)
    {
        if (lx >= r or rx <= l)
            return NULL;
        if (!cur)
            return new Node(0);
        if (l <= lx and rx <= r)
            return new Node(cur->sum);
        int mid = (lx + rx) / 2;
        auto lf = get(l, r, cur->l, lx, mid);
        auto rf = get(l, r, cur->r, mid, rx);
        auto ret = merge(lf, rf);
        return ret;
    }
    int get(int l, int r)
    {
        Node *ret = get(l, r, root, 0, R);
        if (!ret)
            return 0;
        int ret_sum = ret->sum;
        delete ret;
        ret = NULL;
        return ret_sum;
    }
}
```

```

void clear(Node *cur)
{
    if (cur->l)
        clear(cur->l);
    if (cur->r)
        clear(cur->r);
    delete cur;
    cur = NULL;
}

```

7. Strings

7.1 Multiset Hashing

```

int b[2];
int pw[2][N];
int inv[2][N];
void precalc()
{
    b[0] = rand() + 2;
    b[1] = rand() + 2;
    pw[0][0] = pw[1][0] = inv[0][0] = inv[1][0] = 1;
    int invB[] = {modInverse(b[0]), modInverse(b[1])};
    for (int j = 0; j < 2; j++)
        for (int i = 1; i < N; i++)
            pw[j][i] = mul(pw[j][i - 1], b[j]), inv[j][i] = mul(inv[j][i -
1], invB[j]);
}
int doit(char c)
{
    return (c - 'a' + 1);
}
typedef array<int, 2> Hash;
void solve()
{
    int n;
    vector<Hash> prefa(n + 1), prefb(n + 1);
    // A Hashing
    for (int i = 0; i < n; i++)
        for (int j = 0; j < 2; j++)
            prefa[i + 1][j] = addm(prefa[i][j], pw[j][a[i]]);
    // B Hashing
    for (int i = 0; i < n; i++)
        for (int j = 0; j < 2; j++)
            prefb[i + 1][j] = addm(prefb[i][j], pw[j][b[i]]);
}

```

7.2 Rolling Hashing

```

void preHash()
{
    auto now = chrono::high_resolution_clock::now();
    auto duration = chrono::duration_cast<chrono::microseconds>
(now.time_since_epoch());
    srand(duration.count());
    int b[] = {31, 37};
    int pw[2][N];
    int inv[2][N];
    pw[0][0] = pw[1][0] = inv[0][0] = inv[1][0] = 1;
    int invB[] = {modInverse(b[0]), modInverse(b[1])};
    for (int j = 0; j < 2; j++)
        for (int i = 1; i < N; i++)
            pw[j][i] = mul(pw[j][i - 1], b[j]), inv[j][i] = mul(inv[j][i -
1], invB[j]);
}

void rabin()
{
    for (int j = 0; j < 2; j++)
        for (int i = 0; i < n; i++)
            pref[i + 1][j] = addm(pref[i][j], mul(doit(s[i]), pw[j][i]));
    function<Hash(int, int)> getHash = [&](int l, int r) -> Hash {
        Hash h = pref[r];
        h[0] = mul(addm(h[0], -pref[l - 1][0]), inv[0][l - 1]);
        h[1] = mul(addm(h[1], -pref[l - 1][1]), inv[1][l - 1]);
        return h;
    };
}

```

7.3 KMP

```

vector<int> prefixFunction(string pattern)
{
    int n = pattern.size();
    vector<int> pi(n);
    for (int i = 1, j = 0; i < n;)
    {
        if (pattern[i] == pattern[j])
        {
            pi[i] = j + 1;
            i++, j++;
        }
        else
        {
            if (!j)
            {
                pi[i] = 0;
                i++;
            }
        }
    }
}

```

```

        else
        {
            j = pi[j - 1];
        }
    }
}
return pi;
}
vector<int> KMP(string text, string pattern)
{
    vector<int> pi = prefixFunction(pattern);
    vector<int> matches;
    int n = text.size(), m = pattern.size();
    for (int i = 0, j = 0; i < n;)
    {
        if (text[i] == pattern[j])
        {
            i++, j++;
        }
        if (j == m)
        {
            matches.push_back(i - m); // match at position i - m
            j = pi[j - 1]; // continue searching
        }
        else if (i < n && text[i] != pattern[j])
        {
            if (j != 0)
                j = pi[j - 1];
            else
                i++;
        }
    }
    return matches;
}

```

7.4 Trie

```

struct Trie {
    struct Node {
        int children[26]{};
        int f = 0;
    };
    vector<Node> trie;
    Trie() { trie.emplace_back(); }
    void insert(string &s) {
        int node = 0;
        for (auto &i : s) {
            int ch = (i - 'a');
            if (!trie[trie[node].children[ch]].f) {
                trie[node].children[ch] = trie.size();
                trie.emplace_back();
            }
            node = trie[node].children[ch];
        }
        trie[node].f = 1;
    }
};

```

```

        }
        node = trie[node].children[ch];
        trie[node].f++;
    }
}
void erase(string &s) {
    int node = 0;
    for (auto &i : s) {
        int ch = (i - 'a');
        node = trie[node].children[ch];
        trie[node].f--;
    }
}
int query(string &s) {
    int node = 0;
    for (auto &i : s) {
        int ch = (i - 'a');
        if (!trie[trie[node].children[ch]].f) return 0;
        node = trie[node].children[ch];
    }
    return trie[node].f;
}
};

```

7.5 Binary Trie

```

struct BinaryTrie
{
    struct Node
    {
        int children[2]{};
        int f[2]{};
    };
    vector<Node> trie;
    BinaryTrie()
    {
        trie.emplace_back();
        int cur = 0;
        insert(cur);
    }
    void insert(int &x)
    {
        int node = 0, cur;
        for (int i = 30; i >= 0; --i)
        {
            cur = (x >> i) & 1;
            if (trie[node].f[cur] == 0)
            {
                trie[node].children[cur] = trie.size();
                trie.emplace_back();
            }

```

```

        trie[node].f[cur]++;
        node = trie[node].children[cur];
    }
}
void erase(int &x)
{
    int node = 0, cur;
    for (int i = 30; i >= 0; --i)
    {
        cur = (x >> i) & 1;
        trie[node].f[cur]--;
        node = trie[node].children[cur];
    }
}
int query(int &x)
{
    int node = 0, cur, ret = 0;
    for (int i = 30; i >= 0; --i)
    {
        cur = (x >> i) & 1;
        if (trie[node].f[cur ^ 1] == 0)
        {
            node = trie[node].children[cur];
        }
        else
        {
            ret |= (1ll << i);
            node = trie[node].children[cur ^ 1];
        }
    }
    return ret;
}
void clear()
{
    *this = BinaryTrie();
}
};

```

8. Game Theory

8.1 Grundy Numbers

```

const int H = 1e4 + 5;
vector<int> grundy(H, 0);
for (int i = 1; i < H; i++)
{
    set<int> reachable;
    for (auto &move : moves)
        if (i - move >= 0)
            reachable.insert(grundy[i - move]);
}

```

```

    for (auto &x : reachable)
        if (grundy[i] == x)
            grundy[i]++;
        else
            break;
}

```

8.2 MEX in $O(\log(n)^2)$

```

struct Set
{
    map<int, int> mp;
    ordered_set<int> os;
    int mex = 0;
    void insert(int x, int frq)
    {
        mp[x] += frq;
        os.insert(x);
        if (x == mex)
        {
            int l = x, r = mp.rbegin()->first, mid = l + (r - l) / 2;
            while (l <= r)
            {
                mid = l + (r - l) / 2;
                if (os.order_of_key(mid + 1) - os.order_of_key(l) == mid -
l + 1)
                    mex = mid + 1, l = mid + 1;
                else
                    r = mid - 1;
            }
        }
    }
    void insert(int x)
    {
        insert(x, 1);
    }
    bool count(int x)
    {
        if (!mp.count(x))
            return 0;
        return mp[x];
    }
    void erase(int x, int frq)
    {
        mp[x] -= frq;
        if (mp[x] == 0)
            del(x);
    }
    void erase(int x)
    {

```

```
        erase(x, 1);
    }
    void del(int x)
    {
        os.erase(x);
        mp.erase(x);
        if (mex > x)
            mex = x;
    }
};
```

8.3 MEX in $O(\log(n))$

```
int sz;
struct Set
{
    set<int> missing;
    Set()
    {
        for(int i = 0; i < sz; i++)
        {
            missing.insert(i);
        }
    }
    int getMex()
    {
        return missing.empty() ? sz : *missing.begin();
    }
    void insert(int x)
    {
        if(missing.count(x))
            missing.erase(x);
    }
    void clear()
    {
        for(int i = 0; i < sz; i++)
        {
            missing.insert(i);
        }
    }
};
```

Some properties of bitwise operations:

- $a|b = a \oplus b + a \& b$
- $a \oplus (a \& b) = (a|b) \oplus b$
- $b \oplus (a \& b) = (a|b) \oplus a$
- $(a \& b) \oplus (a|b) = a \oplus b$

Addition:

- $a+b = a|b + a \& b$
- $a+b = a \oplus b + 2(a \& b)$

Subtraction:

- $a-b = (a \oplus (a \& b)) - ((a|b) \oplus a)$
- $a-b = ((a|b) \oplus b) - ((a|b) \oplus a)$
- $a-b = (a \oplus (a \& b)) - (b \oplus (a \& b))$
- $a-b = ((a|b) \oplus b) - (b \oplus (a \& b))$

Hope you like it. Please let me know if you find any issues. **All of these properties are elaborated in this [blog here](#). You can also check it out to learn more!**

✅ Final Formula.

$$\text{Area} = r^2 \cos^{-1} \left(\frac{d^2 + r^2 - R^2}{2dr} \right) + R^2 \cos^{-1} \left(\frac{d^2 + R^2 - r^2}{2dR} \right) - \frac{1}{2} \sqrt{(-d+r+R)(d+r-R)(d-r+R)(d+r+R)}$$